

AI For Math Seminar, ZJU

Lec 03: Lean AI — instance

Mulikas

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

Mathlib AI

- **instance** Lean
- **Mathlib**
- **AI** LLM Lean

— instance Lean

```
G : Type*   [Group G]   mul_assoc
              Lean
```

-
- [...] Lean instance
- typeclass

```
variable (G : Type*) [Group G]

#synth Group G

#check mul_assoc

#check inv_mul_cancel
```

- mul, one, inv
- add, zero, neg
- Hom, id, comp

-
-
-

goal

```
#print      := by
```

Outline

- 1
- 2 AddGroup / Group**
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

AddGroup / Group — AddGroup Group

AddGroup A	0, +, -a	zero, add, neg
Group G	1, *, g^{-1}	one, mul, inv

```
#check AddGroup
```

```
#check Group
```

```
#check neg_add_cancel
```

```
#check inv_mul_cancel
```

```
example (A : Type*) [AddGroup A] (a : A) :  
  -a + a = 0 := neg_add_cancel a
```

```
example (G : Type*) [Group G] (g : G) :  
  g1 * g = 1 := inv_mul_cancel g
```

```
class / / Lean /
```

— AddGroup

- [Group G]
- $a * b$
- `inv_mul_cancel`
-
- [AddCommGroup V]
- $u + v$
- `neg_add_cancel`
- [Module $K\ V$]

AddGroup

Monoid — Group

mul	$G \rightarrow G \rightarrow G$	
one		
inv	$G \rightarrow G$	
mul_assoc	cases /	/ group
one_mul, mul_one	rfl	
inv_mul_cancel		

- mul, one, inv —
- mul_assoc, one_mul, mul_one, inv_mul_cancel —
- Lean instance

Sign —

```
inductive Sign where
  | pos
  | neg
  deriving DecidableEq, Repr
```

```
open Sign
```

```
instance : Group Sign where
  mul a b :=
    match a, b with
    | pos, x => x
    | neg, pos => neg
    | neg, neg => pos
  one := pos
  inv a := a
  -- laws come next
```

- pos
- $\text{neg} * \text{neg} = \text{pos}$
-
-

```
pos * neg
neg * neg
```

```

mul_assoc := by
  intro a b c
  cases a <;> cases b
    <;> cases c <;> rfl
one_mul := by
  intro a
  cases a <;> rfl
mul_one := by
  intro a
  cases a <;> rfl
inv_mul_cancel := by
  intro a
  cases a <;> rfl

```

```
#synth Group Sign
```

```

example (a : Sign) :
  a1 * a = 1 := by
  exact inv_mul_cancel a

```

- ① intro
- ② cases pos neg
- ③ rfl
- ④ Group

+ cases + rfl

=

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

— Subtype

```
def NonzeroInt :=  
  {n : // n 0}  
  
example : NonzeroInt :=  
  1, by norm_num  
  
example (x : NonzeroInt) : :=  
  x.1  
  
example (x : NonzeroInt) :  
  x.1 0 := x.property  
  
#check Subtype.val  
#check Subtype.property
```

- x.1
- x.2 x.property
- value, proof
-

Subtype " "

Group — Subgroup = carrier +

```
def oneSubgroup (G : Type*)
  [Group G] : Subgroup G
  where
    carrier := {g | g = 1}
    one_mem' := by rfl
    mul_mem' := by
      intro a b ha hb
      rw [ha, hb]; simp
    inv_mem' := by
      intro a ha
      rw [ha]; simp

example (G : Type*) [Group G]
  (x : oneSubgroup G) :
  (x : G) = 1 := x.property
```

- 1 carrier g : G
- 2 one_mem'
- 3 mul_mem'
- 4 inv_mem'
- 5 Mathlib

Subgroup
Lean

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 **Ring / CommRing**
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

Ring / CommRing — Ring

Lean

0, +, -, / /	AddCommGroup
1, *, /	Monoid
0 0	left_distrib, right_distrib
	CommRing / mul_comm

Ring

PairInt — Ring instance

```
@[ext]
structure PairInt where
  x : Int
  y : Int

def equivProd : PairInt → Int × Int
  where
    toFun p      := (p.x, p.y)
    invFun q     := q.1, q.2
    left_inv     := by intro p; ext
                  <;> rfl
    right_inv    := by intro q; cases
                  q; rfl

instance : CommRing PairInt :=
  equivProd.commRing
```

- $\text{Int} \times \text{Int}$
- PairInt
- Equiv.commRing
-

#check — Ring

```
#synth CommRing PairInt
#check add_assoc
#check neg_add_cancel
#check left_distrib
#check right_distrib
#check mul_assoc
#check mul_comm
example (p q r : PairInt) :
  p * (q + r) = p * q + p * r :=
  by
    exact mul_add p q r
example (p q : PairInt) :
  (p + q)^2 = p^2 + 2*p*q + q^2
  := by
  ring
```

① #synth

② #check

③

④ ring

Ring — Subring Subgroup

```
def diagonalSubring : Subring ( ×
  ) where
  carrier := {p | p.1 = p.2}
  zero_mem' := by simp
  one_mem' := by simp
  add_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  neg_mem' := by
    intro a ha; simp at ha ; rw
      [ha]
  mul_mem' := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
example (x : diagonalSubring) :
  (x : × ).1 = (x : × ).2 :=
  x.property
```

- $p.1 = p.2$ pair
- zero_mem'
- one_mem'
- add_mem', neg_mem', mul_mem'
- x.property
-

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field**
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

— Field = +

			Lean
CommRing K			ring
Nontrivial K			0 1
inv, div			$x / y = x * y^{-1}$
mul_inv_cancel			$x \neq 0$
inv_zero	Lean	$0^{-1} = 0$	
AI	$x / x = 1$	$x \neq 0$	

```
@[ext]
structure WrappedQ where
  val : Rat

def equivRat : Equiv WrappedQ Rat
  where
    toFun x    := x.val
    invFun q    := { val := q }
    left_inv    := by intro x; ext; rfl
    right_inv   := by intro q; rfl

instance : Field WrappedQ :=
  equivRat.field
```

- Rat
-
- field_simp [hx]
- statement

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule**
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

Lean

```

[Semiring R]  [Field K]
[AddCommGroup V]
r • v        [Module R V]
smul_add, add_smul, mul_smul, one_smul

```

```

variable {K : Type*} [Field K]
variable {V : Type*} [AddCommGroup V] [Module K V]
#check smul_add      -- r • (v + w) = r • v + r • w
#check add_smul      -- (r + s) • v = r • v + s • v
#check mul_smul      -- (r * s) • v = r • (s • v)
#check one_smul      -- (1 : K) • v = v

```



```

def diagonalSubmodule :
  Submodule ( × ) where
  carrier      := {p | p.1 = p.2}
  zero_mem'    := by simp
  add_mem'     := by
    intro a b ha hb
    simp at ha hb ; rw [ha, hb]
  smul_mem'    := by
    intro c p hp
    change (c * p.1 = c * p.2)
    rw [hp]
example (x : diagonalSubmodule) :
  (x : × ).1 = (x : × ).2 :=
  x.property

```

- ① carrier
- ② zero_mem'
- ③ add_mem'
- ④ smul_mem'
- ⑤

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

Symplectic vector space —

```
variable (F W : Type*)
variable [Field F] [Fintype F]
variable [AddCommGroup W] [Module
  F W]
variable [FiniteDimensional F W]

def IsAlternating
  ( : W → [F] W → [F] F) : Prop :=
  w : W,   w w = 0

def IsNondegenerate
  ( : W → [F] W → [F] F) : Prop :=
  w : W, ( v : W,   w v = 0) → w =
    0
```

• W

•

• Prop

•

Al

Lean

CompletePolarization

```
def TotallyIsotropic
  ( : W → [F] W → [F] F)
  (U : Submodule F W) : Prop :=
  u U, v U, u v = 0

structure CompletePolarization
  ( : W → [F] W → [F] F) where
  X      : Submodule F W
  Y      : Submodule F W
  isotropic_X : TotallyIsotropic F
    W X
  isotropic_Y : TotallyIsotropic F
    W Y
  disjoint    : Disjoint X Y
  sup_eq_top  : X Y =
```

- X, Y
-
- $\text{Disjoint } X \ Y$
- $X \ Y =$

$\text{sup_eq_top} \quad w : W$

存在性先闭合, `Classical.choose` 才能取对象

```
lemma exists_decomposition
  (P : CompletePolarization F W  $\omega$ ) (w : W) :
   $\exists x, x \in P.X \wedge \exists y, y \in P.Y \wedge w = x + y$ 
   $\hookrightarrow$  := by
  have hw : w  $\in$  sup P.X P.Y := by
    rw [P.sup_eq_top]
    exact Submodule.mem_top
  rcases
    Submodule.mem_sup.mp hw with
    (x, hx, y, hy, hxy)
  exact (x, hx, y, hy, hxy.symm)

noncomputable def xComponent (w : W) : W :=
  Classical.choose (exists_decomposition F W P
 $\hookrightarrow$  w)
```

- `sup_eq_top` 把任意 w 送入 $X \sqcup Y$
- `Submodule.mem_sup.mp` 给出具体分解
- `choose` 只在存在性证明完成后使用
- 这里不再保留 `sorry`

choose_spec

```
noncomputable def yComponent
  (P : CompletePolarization F W )
  (w : W) : W :=
  Classical.choose
    (Classical.choose_spec
      (exists_decomposition F W P
        w)).2
```

```
lemma decompose_by_components
  (P : CompletePolarization F W )
  (w : W) :
  w = xComponent F W P w
    + yComponent F W P w := by
  dsimp [xComponent, yComponent]
  exact
    (Classical.choose_spec
      (Classical.choose_spec
        (exists_decomposition F W
          P w)).2)).2
```

- $x \quad P.X$
- $y \quad P.Y$
- y
- $w = x + y$

Lean witness
choose_spec

选择给出分量, Disjoint 给出唯一性

目标

```
#check xComponent_add F W ω P
-- (w1 w2 : W) →
--   xComp (w1 + w2) = xComp w1 + xComp w2
```

比较两个分解

- $w_1 + w_2 = x_{12} + y_{12}$
- $w_1 + w_2 = (x_1 + x_2) + (y_1 + y_2)$
- 两式相减得到一个同时属于 X 与 Y 的向量

要点: 选择公理只给 witness; 线性来自分解的唯一性, 而不是来自 choose 本身。

用直和消去差异

- $P.\text{disjoint.le_bot}$:
 $X \cap Y \leq \perp$
- 差向量只能是 0
- 因而 $x_{12} = x_1 + x_2$

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8**
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16


```

class CategoryStruct (obj :
  Type u)
  extends Quiver obj where
id :   X : obj, Hom X X
comp : {X Y Z : obj},
  (X → Y) → (Y → Z) → (X → Z)
class Category (obj : Type u)
  extends CategoryStruct
    obj where
id_comp : {X Y} (f : X
  → Y),
  X → f = f
comp_id : {X Y} (f : X
  → Y),
  f → Y = f
assoc : f ∘ g ∘ h,
  (f ∘ g) ∘ h = f ∘ (g ∘ h)

```

- Hom Quiver
- $X \rightarrow id \rightarrow X$
- $f \circ g$
- instance
-

```

structure Functor (C : Type u)
  [Category C]
  (D : Type u) [Category D]
  where
    obj : C → D
    map : {X Y : C},
      (X → Y) → (obj X → obj Y)
    map_id : 1_X,
      map (1_X) = 1_{obj X}
    map_comp : f ∘ g,
      map (f ∘ g) = map f ∘ map g

```

- $F.obj \ X$
- $F.map \ f$
- $F \ \mathcal{C} \ \mathcal{D}$
- $F : \mathcal{C} \rightarrow \mathcal{D}$

```

structure NatTrans (F G : C
  D) where
  app (X : C) : F.obj X
    G.obj X
  naturality X Y : C (f : X
    Y) :
    F.map f    app Y =
    app X    G.map f

```

• .app X

•

•

Mathlib morphism : F G

```

structure HomologicalComplex
  (c : ComplexShape) where
  X :  $\rightarrow V$ 
  d :  $i \rightarrow j, X \ i \rightarrow X \ j$ 
  shape :  $i \rightarrow j$ ,
     $\neg c.Rel \ i \ j \rightarrow d \ i \ j = 0$ 
  d_comp_d' :  $i \rightarrow j \rightarrow k$ ,
     $c.Rel \ i \ j \rightarrow c.Rel \ j \ k \rightarrow$ 
       $d \ i \ j \rightarrow d \ j \ k = 0$ 

```

- $X \ i \rightarrow i$
- $d \ i \rightarrow j$
- shape
- d_comp_d'

ComplexShape.up

HomologicalComplex.d_comp_d

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8**
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

Category instance

```
open CategoryTheory
inductive OneObj where
  | star
instance : Category OneObj where
  Hom _ _ := PUnit
  id _ := PUnit.unit
  comp _ _ := PUnit.unit
  id_comp := by
    intro X Y f; cases f; rfl
  comp_id := by
    intro X Y f; cases f; rfl
  assoc := by
    intro W X Y Z f g h
    cases f; cases g; cases h; rfl
```

① Hom/id/comp

②

③

④

HomologicalComplex

```
open CategoryTheory
open CategoryTheory.Limits
noncomputable def tinyZeroComplex :
  HomologicalComplex
  (Discrete PUnit)
  (ComplexShape.up ) where
X _ := Discrete.mk PUnit.unit
d _ _ := 0
shape := by
  intro i j hij; rfl
d_comp_d' := by
  intro i j k hij hjk; simp
```

- Discrete PUnit
- 0
- shape
- simp

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass**
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

instance

instance

- Type
-
- Group G , Field K
-

instance

- Prop
-
- Fact P , [Decidable P]
-

```

-- Fact P wraps a proof of P as an
  instance
-- class Fact (P : Prop) : Prop
  where
--   out : P

-- Usage: bring a hypothesis into
  instance search
example (P : Prop) [Fact P] : P :=
  Fact.out

-- haveI temporarily introduces an
  instance
example (n : ) (h : n 0) : 0 < n
  := by
  haveI : Fact (n 0) := h
  exact Nat.pos_of_ne_zero Fact.out

```

- ① Fact P P
- ② Fact.out
- ③ [Fact ...]
- ④ haveI

命题 class 也可以声明具体 instance

```
class IsEven (n : Nat) : Prop where
  witness :  $\exists k, n = 2 * k$ 

instance : IsEven 8 where
  witness := (4, by norm_num)

example :  $\exists k, 8 = 2 * k$  := by
  exact (inferInstance : IsEven 8).witness
```

- `IsEven 8 : Prop`
是性质，不是新数据结构
- `instance` 把证明放入 typeclass search
- `inferInstance` 找到证明对象
- `witness` 取出数学内容

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search**
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

#synth、infer_instance 与 haveI

-- 询问：环境中能否合成实例？

```
#synth Group Sign
```

-- 在证明中直接让 Lean 合成

```
example : Group Sign := by
  infer_instance
```

-- 把普通假设临时包装成局部 instance

```
example (n : Nat) (h : n ≠ 0) : 0 < n := by
  haveI : Fact (n ≠ 0) := ⟨h⟩
  exact Nat.pos_of_ne_zero (Fact.out : n ≠
    ↪ 0)
```

- #synth 用于交互检查
- infer_instance 产生实例项
- haveI 只在当前证明作用域有效
- 不要临时引入另一套带运算的群结构，否则记号可能指向不同实例

检索实战：先让 Lean 提供候选，再写稳定证明

```
#check Finset.card_range

example (n : Nat) : (Finset.range n).card =
  ↪ n := by
  exact?
-- suggestion: exact Finset.card_range n

example (G : Type*) [Group G] (a : G) :
   $a^{-1} * a = 1$  := by
  apply?
-- suggestion: exact inv_mul_cancel a

example (n : Nat) :
  (Finset.range (n + 1)).card = n + 1 :=
  ↪ by
  simp?
```

- #check 验证名字与类型
- exact?/apply? 搜索可直接使用的定理
- simp? 返回建议的化简集合
- 定稿时保留短而明确的 proof，避免依赖偶然搜索结果

```

-- mul_assoc needs only Semigroup
#check @mul_assoc
--   {} [Semigroup ] (a b c : ),
--   a * b * c = a * (b * c)

-- inv_mul_cancel needs Group
#check @inv_mul_cancel
--   {G} [Group G] (a : G),
--   a1 * a = 1

-- group tactic works for any Group
example {G : Type*} [Group G]
  (a b : G) :
  (a * b)1 = b1 * a1 := by
  group

```

-
- [Group G]
[CommGroup G]
- AI
- #check

— Equiv instance

```
-- Transfer CommRing along an
   equivalence
def equivProd : PairInt    Int × Int
  := ...
instance : CommRing PairInt :=
  equivProd.commRing

-- Transfer Field along an
   equivalence
def equivRat : WrappedQ    := ...
instance : Field WrappedQ :=
  equivRat.field
```

-
- equivalence
- wrapper subtype
- AI Lean

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean**
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

statement

```
--      Missing hypothesis: what if  $x$   
      = 0?
```

```
example (K : Type*) [Field K] (x :  
  K) :  
  x / x = 1 := by  
  field_simp          -- fails!
```

```
--      Add the necessary assumption
```

```
example (K : Type*) [Field K]  
  (x : K) (hx : x ≠ 0) :  
  x / x = 1 := by  
  field_simp [hx]
```

- $x = 0$
- Lean
- tactic
- statement proof

```

--      False in a non-commutative
      group!
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = a1 * b1 := by
group      -- fails: not provable

--      Correct for any Group
example (G : Type*) [Group G] (a b
  : G) :
  (a * b) 1 = b1 * a1 := by
group

--      Commutative version needs
      CommGroup
example (G : Type*) [CommGroup G]
  (a b : G) :
  (a * b) 1 = a1 * b1 := by
group

```

-
- group
- [CommGroup G]
-

```
--      This theorem name does not
      exist
example (n : ) :
  (Finset.range n).card = n := by
    exact Finset.card_range_eq_n --
      Error!

--      Check the real API first
#check Finset.card_range
--      Finset.card_range :
--      (n : ), (Finset.range
      n).card = n

example (n : ) :
  (Finset.range n).card = n :=
    Finset.card_range n
```

-
- API
- AI #check
- LeanSearch / Loogle

decide

```
--      decide requires [Decidable P]
example (P : Prop) : P  ¬ P := by
  decide      -- Error: no Decidable
              instance

--      Constructive: add [Decidable
P]
example (P : Prop) [Decidable P] :
  P  ¬ P := by
  by_cases h : P
  · exact Or.inl h
  · exact Or.inr h

--      Classical: use em
example (P : Prop) : P  ¬ P := by
  classical; exact em P
```

- ① decide
[Decidable P]
- ② decide
- ③ classical
- ④ AI

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI**
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

	AI	Lean
Statement	typeclass	
Search	#check	theorem
Proof	proof	lemma
Review		simp
AI	Lean kernel	

Prompt

Lean 4 / Mathlib
theorem statement

1. typeclass
- 2.
3. #check
4. theorem
5. statement

AI 代码实战：结构假设决定 tactic 能做什么

交换环: `rw + ring`

```
example {R : Type*} [CommRing R]
  (a b c d : R)
  (h1 : c = d * a + b)
  (h2 : b = a * d) :
  c = 2 * a * d := by
  rw [h1, h2]
  ring
```

线性有序环: `linarith`

```
example (x y : Real)
  (h1 : x + y = 10)
  (h2 : x - y = 4) :
  x = 7 := by
  linarith
```

- AI 可以起草 `statement` 和 `proof`, 但必须明确 `CommRing`、`Real` 等结构
- `ring` 做多项式恒等式归一化; `linarith` 使用线性等式与不等式假设

	$x / x = 1$	$x \cdot 0$	AI
	Group	CommGroup	#check
API	theorem		#check
			Mathlib
	tactic		lemma proof

AI 形式化的前景取决于“检索 – 编译 – 审核”闭环

近期可可靠加速

- 从自然语言生成 theorem statement 草案
- 根据目标类型检索 Mathlib 定理
- 把长证明拆成局部 lemma
- 根据编译错误修复 API 与 instance

研究级潜力

- 大规模 autoformalization 与形式化语料构建
- 交互式证明搜索和跨文件重构
- 让形式化库成为可计算的数学知识库
- 研究论文中的定义、引理和依赖可机器检查

仍需解决

语义对齐、库版本变化、长上下文依赖和证明可维护性。当前最有效的方法是受约束 prompt、检索增强、短编译循环与人工数学审核。

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean**
- 14 Finset
- 15 SimpleGraph
- 16

Lean

- theorem statement
- proof term
- instance
-

Lean

-
- Mathlib
-
-

更深入：观察 elaborator 的 instance 搜索

```
set_option trace.Meta.synthInstance true in
#synth Module  $\mathbb{Q}$  ( $\mathbb{Q} \times \mathbb{Q}$ )
```

-- 查看定理依赖了哪些公理

```
#print axioms LeanZjuD3.Ch10.xComponent_add
```

- trace 展示父类实例如何递归合成
- elaborator 把记号、隐式参数和 tactic 变成 proof term
- kernel 只检查最终 proof term 的类型
- #print axioms 会揭示 Classical.choice 等非构造依赖

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset**
- 15 SimpleGraph
- 16

Finset α

-
- card
- range n $\{0, \dots, n - 1\}$
- filter
- #check
Finset.card_range
- simp
- native_decide
- sum_range_succ


```
open BigOperators Finset

-- card of range n is n (general)
#check Finset.card_range
-- (n : ℕ), (Finset.range n).card
   = n
example (n : ℕ) :
  (Finset.range n).card = n :=
  Finset.card_range n

-- constant sum over range
  (general)
example (n : ℕ) :
  ∑ i in Finset.range n, (1 : ℕ) =
    n := by
  simp

-- counting evens in {0,...,9}
  (concrete)
example :
  ((Finset.range 10).filter
    fun n => n % 2 = 0).card = 5
    := by
  decide
```

- API



高斯求和：先消去自然数除法，再归纳

```
theorem gauss_sum (n : Nat) :  
  (∑ i in Finset.range (n + 1), i) =  
    n * (n + 1) / 2 := by  
  symm  
  apply Nat.div_eq_of_eq_mul_right  
    (by norm_num : 0 < 2)  
  induction n with  
  | zero => simp  
  | succ n ih =>  
    rw [Finset.sum_range_succ, mul_add 2, ←  
      ↪ ih]  
    ring
```

- 先把 $/2$ 转成乘法等式
- `sum_range_succ` 拆出最后一项
- 归纳假设替换旧和式
- `ring` 完成剩余恒等式

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph**
- 16

— SimpleGraph = +

Lean

```
G : SimpleGraph V V
G.Adj u v          u v
symm
loopless            G.Adj v v
V = Fin n          fin_cases
```

```

def path3 : SimpleGraph (Fin 3)
  where
    Adj i j :=
      (i = 0    j = 1)  (i = 1    j =
        0)
      (i = 1    j = 2)  (i = 2    j = 1)
    symm := by
      intro i j h
      fin_cases i <;> fin_cases j <;>
      simp_all
    loopless := by
      intro i h
      fin_cases i <;> simp_all

```

- 0, 1, 2
- 0--1 1--2
- symm
- loopless

```
-- General: adjacency implies
distinct vertices
```

```
example (V : Type*) (G :
  SimpleGraph V)
  (u v : V) (h : G.Adj u v) : u
    v :=
  G.ne_of_adj h
```

```
-- Concrete: path3 adjacency
```

```
example : path3.Adj 0 1 := by simp
  [path3]
```

```
example : path3.Adj 1 2 := by simp
  [path3]
```

```
example : ¬ path3.Adj 0 2 := by
  simp [path3]
```

```
-- General: no vertex is adjacent
to itself
```

```
example (V : Type*) (G :
  SimpleGraph V)
  (v : V) : ¬ G.Adj v v :=
  G.loopless v
```

- ① Adj
- ②
- ③
- ④ simp / decide

SimpleGraph.top 的邻接关系可由 simp 展开

```
#check @SimpleGraph.ne_of_adj

example :
  path3.Adj 0 1 ∧ ¬ path3.Adj 0 2 := by
  constructor
  · simp [path3]
  · simp [path3]

example (V : Type*) [DecidableEq V]
  (u v : V) (h : u ≠ v) :
  (Top.top : SimpleGraph V).Adj u v := by
  simpa using h
```

- `ne_of_adj`: 邻接推出顶点不同
- 展开 `path3` 后, `simp` 判断具体边
- 对完全图 `top`, 邻接正是 $u \neq v$
- 不再引用不存在的 `top_adj`

Outline

- 1
- 2 AddGroup / Group
- 3 Subtype
- 4 Ring / CommRing
- 5 Field
- 6 Module / Submodule
- 7
- 8
-
- 9 Prop-valued Typeclass
- 10 Instance Search
- 11 AI Lean
- 12 AI
- 13 Lean
- 14 Finset
- 15 SimpleGraph
- 16

instance

- ① `G / R / K / C`
- ② `AddGroup Group CommRing Field Module Category`
- ③ `#print`
- ④ `carrier`
- ⑤ `add/mul/zero/one/smul/Hom/id/comp`
- ⑥ `#synth`
- ⑦ `#check`
- ⑧ `Al Lean`